ML101
Homework 05: A 'gentle' Introduction to Self-Organizing Maps (Kohonen Networks) in R.

Andrés Oswaldo Calderón Romero, Ph.D.

November 12, 2026

1 Introduction

Welcome to Homework 05: *The Neural Cartographer*. In this practical session, we will expand our unsupervised and semi-supervised learning toolkit by exploring **Self-Organizing Maps (SOMs)**, where you will take 13-dimensional Italian wine chemistry and project it onto a two-dimensional topological map without losing the underlying structure—or the vintage.

Created by Finnish professor Teuvo Kohonen in the 1980s[1], a Self-Organizing Map is an unsupervised artificial neural network designed for nonlinear dimensionality reduction, clustering, and high-dimensional visualization. Unlike deep learning architectures that often act as inscrutable black boxes, a SOM behaves like an **elastic fishing net** cast into a high-dimensional cloud of observations. During training, the knots of the net (neurons or nodes) compete to capture nearby data points, pulling their neighboring knots along with them until the 2D grid stretches and folds around the natural geometry of your dataset.

This laboratory is structured into two main phases. Initially, you will work through a five-part guided walkthrough in R using the classic `kohonen`[2] and `aweSOM` packages. Following the guided tutorial, you will transition to the autonomous portion of the lab, where you will source a multivariate dataset from the UC Irvine Machine Learning Repository and perform a complete topological mapping and clustering analysis.

Table 1 summarizes the primary R packages that form the modern Self-Organizing Map ecosystem.

2 Homework Phase 1: Prepping the Cellar (Data & Feature Scaling)

Our guided mission is to map $N = 177$ bottles of Italian wine originating from three grape cultivars (*Barolo*, *Grignolino*, and *Barbera*) characterized across $p = 13$ chemical measurements—without revealing the cultivar labels to the network during training. Conveniently, this classic dataset is bundled directly inside the `kohonen` package.

2.1 Why You Must Scale First

If you feed raw wine chemistry directly into a SOM, the variable `proline` (an amino acid measured in hundreds of milligrams) will completely dominate `nonflavanoid phenols` (measured in tenths of a

[1] Kohonen, T. (2001). *Self-Organizing Maps* (3rd ed.). Springer. https://doi.org/10.1007/978-3-642-56927-2

[2] Wehrens, R., & Kruisselbrink, J. (2018). Flexible Self-Organizing Maps in kohonen 3.0. *Journal of Statistical Software*, 87(7), 1–18. https://www.jstatsoft.org/article/view/v087i07

Table 1: The R Ecosystem for Self-Organizing Maps

Package	Role in Workflow	Best Feature	Documentation
`kohonen`	Core training engine	Fast C++ backend; supports standard SOMs (`som`), supervised fusion (`xyf`), and multi-block layers (`supersom`).	CRAN
`aweSOM`	Interactive diagnostics	Generates interactive star/radar plots and computes topological and quantization quality metrics automatically.	CRAN
`SOMbrero`	Categorical & relational	Handles contingency tables, factors, and dissimilarity matrices (ideal for text or survey networks).	CRAN
`popsom`	Smooth U-Matrix terrain	Builds high-resolution, topographic "starburst" heatmaps with cluster centroids.	CRAN [3]

milligram). Because SOM competition relies on Euclidean distance in $\mathbb{R}^p$:

$$d(\mathbf{x}, \mathbf{w}_j) = \sqrt{\sum_{k=1}^{p} (x_k - w_{jk})^2}, \tag{1}$$

unscaled variables with large numeric ranges monopolize the distance calculation. **Failing to standardize features is the number one cause of distorted Self-Organizing Maps.**

```r
# Install required packages if not yet installed
# install.packages(c("kohonen", "aweSOM"))

library(kohonen)
library(aweSOM)

# 1. Load the built-in Italian wine dataset
data(wines)
head(wines[, 1:6])

# Note: 'vintages' is a separate factor vector loaded with data(wines)
# containing the true grape labels (Barolo, Grignolino, Barbera).
table(vintages)

# 2. Standardize every variable to mean = 0 and standard deviation = 1
wines_scaled <- scale(wines)
```

3 Homework Phase 2: Weaving the Grid & Training the Brain

To construct a SOM, we first define the two-dimensional arena where the neurons reside using `somgrid()`. Two architectural decisions govern how the map behaves:

1. **Topology (`hexagonal` vs. `rectangular`):** Always prefer `hexagonal`. On a rectangular grid, diagonal neighbors sit at a distance of $\sqrt{2} \approx 1.41$ units whereas horizontal and vertical neighbors sit at

1.0 unit, introducing directional bias. On a hexagonal grid, every interior neuron sits at an identical unit distance from all six of its immediate neighbors.

2. **Grid Dimensions:** A widely used heuristic proposed by Vesanto is to allocate approximately $5\sqrt{N}$ total neurons, or to aim for roughly 5 to 10 observations per node. With $N = 177$ wines, a 5×5 grid (25 neurons, averaging ≈ 7 bottles per neuron) balances resolution and generalization.

3.1 The Competitive Learning Loop

At each training iteration t, an observation $\mathbf{x}$ is presented to the grid:

- **Competition:** Every neuron j compares its internal weight vector $\mathbf{w}_j$ (called a *codebook vector*) to $\mathbf{x}$. The neuron minimizing $d(\mathbf{x}, \mathbf{w}_j)$ wins and is crowned the **Best Matching Unit (BMU)**, denoted c.

- **Cooperation & Adaptation:** The BMU and its topological neighbors update their codebook vectors toward $\mathbf{x}$ according to:

$$\mathbf{w}_j(t+1) = \mathbf{w}_j(t) + \alpha(t)\, h_{cj}(t)\, [\mathbf{x} - \mathbf{w}_j(t)]\,, \tag{2}$$

  where $\alpha(t)$ is the monotonically decreasing learning rate and $h_{cj}(t)$ is the neighborhood kernel function that shrinks in radius over time.

```r
# Set a random seed for reproducible weight initialization
set.seed(42)

# 1. Define a 5x5 hexagonal grid
# toroidal = FALSE keeps map edges bounded rather than wrapping around
wine_grid <- somgrid(xdim = 5, ydim = 5, topo = "hexagonal", toroidal = FALSE)

# 2. Train the Self-Organizing Map
wine_som <- som(
  X = wines_scaled,
  grid = wine_grid,
  rlen = 500,                 # Number of complete passes through the dataset
  alpha = c(0.05, 0.01),      # Learning rate decays linearly from 0.05 to 0.01
  keep.data = TRUE            # Retain scaled data inside object for diagnostics
)

summary(wine_som)

# 3. Diagnostic 1: Verify training convergence
plot(wine_som, type = "changes", main = "SOM Training Progress (Mean Distance to BMU)")
```

Before inspecting any hexagonal heatmaps, always examine the learning curve produced by `type = "changes"`. As training progresses across the 500 epochs (`rlen`), the mean distance to the closest unit should drop steeply and settle into a flat plateau. If the curve is still descending at the right edge, increase `rlen`.

4 Homework Phase 3: Reading the Map (The Five Essential Plots)

The `kohonen` package overloads R's base `plot()` generic with a comprehensive suite of diagnostic visualizations. Let us first define a high-contrast cool-to-warm color palette function so that low values appear in cool blue tones and high values appear in warm red tones.

```
# Define a custom cool-to-warm palette (Blue = Low, Yellow = Mid, Red = High)
cool_warm <- function(n, alpha = 1) {
  hcl.colors(n, palette = "Roma", alpha = alpha, rev = TRUE)
}
```

4.1 Population Census (`counts`) and Node Quality (`quality`)

- **Node Counts (`type = "counts"`):** Displays how many wine bottles were assigned to each hexagon. Ideally, samples distribute fairly evenly across the grid. Empty grey nodes represent "dead neurons" (indicating an oversized grid), whereas a single node holding 40+ samples suggests that region needs finer resolution.
- **Node Quality (`type = "quality"`):** Displays the mean quantization distance between the observations mapped to a node and that node's codebook vector. Cool colors indicate tight, homogeneous clusters; hot spots highlight neurons forced to summarize heterogeneous bottles.

```
par(mfrow = c(1, 2))

plot(wine_som, type = "counts", palette.name = cool_warm,
     main = "Node Counts (Sample Population)")

plot(wine_som, type = "quality", palette.name = cool_warm,
     main = "Node Quality (Mean Distance)")

par(mfrow = c(1, 1))
```

4.2 The U-Matrix (`dist.neighbours`) and Neuronal DNA (`codes`)

- **Unified Distance Matrix (`type = "dist.neighbours"`):** The U-Matrix measures the average Euclidean distance between each neuron and its immediate hexagonal neighbors. Interpret it like a topographic elevation map: warm/dark colors represent steep **mountain ridges** (large distances between adjacent nodes) that form natural boundaries between clusters, whereas cool colors represent flat **valleys** of closely related neurons.
- **Codebook Profiles (`type = "codes"`):** Renders a miniature fan or segment chart inside each hexagon displaying the magnitude of all 13 chemical features for that neuron's prototype recipe.

```
par(mfrow = c(1, 2))

# The U-Matrix: Identify mountain ridges separating valley basins
```

```r
plot(wine_som, type = "dist.neighbours", palette.name = cool_warm,
     main = "U-Matrix (Neighbor Distances)")

# The Codebook Vectors: Feature profile of each neuron
plot(wine_som, type = "codes", codeRendering = "segments",
     main = "Neuron Profiles (Codebook Vectors)")

par(mfrow = c(1, 1))
```

4.3 Single-Variable Component Planes (`property`)

To inspect how individual chemical properties distribute across the map, extract columns from the codebook matrix using `getCodes(wine_som)` and plot each variable with `type = "property"`. Note that in `kohonen` version 3.0 and above, codebook matrices are stored in a list to support multi-layer networks, so calling `getCodes()` is much safer than indexing `wine_som$codes` directly.

```r
# Safely extract the 25 x 13 codebook matrix
codes_matrix <- getCodes(wine_som)

# Compare Alcohol, Flavanoids, and Color Intensity across the map
par(mfrow = c(1, 3))

plot(wine_som, type = "property", property = codes_matrix[, "alcohol"],
     palette.name = cool_warm, main = "Alcohol Level")

plot(wine_som, type = "property", property = codes_matrix[, "flavanoids"],
     palette.name = cool_warm, main = "Flavanoids")

plot(wine_som, type = "property", property = codes_matrix[, "col.int"],
     palette.name = cool_warm, main = "Color Intensity")

par(mfrow = c(1, 1))
```

5 Homework Phase 4: Finding the Tribes (Clustering & Validation)

A 25-neuron grid is ideal for visual exploration, but downstream decision-makers often require a discrete number of macro-clusters K. Rather than clustering noisy raw observations directly, we apply **Ward's Hierarchical Agglomerative Clustering** to the 25 smoothed codebook vectors and carve territory borders directly onto the SOM grid.

```r
# 1. Compute pairwise Euclidean distance between the 25 neuron codebook vectors
som_dist <- dist(getCodes(wine_som))

# 2. Run Ward's hierarchical clustering on the neurons
som_hc <- hclust(som_dist, method = "ward.D2")

# 3. Cut the dendrogram into k = 3 macro-clusters
som_clusters <- cutree(som_hc, k = 3)
```

```r
# 4. Plot the SOM colored by cluster and overlay true grape cultivar numbers
cluster_palette <- c("#E69F00", "#56B4E9", "#009E73")

plot(wine_som, type = "mapping",
     bgcol = cluster_palette[som_clusters],
     labels = as.integer(vintages), # 1 = Barbera, 2 = Barolo, 3 = Grignolino
     col = "black",
     main = "Unsupervised SOM Clusters vs. True Cultivars")

# 5. Draw thick cluster boundaries around the discovered territories
add.cluster.boundaries(wine_som, som_clusters, lwd = 4)

# 6. Validate unsupervised cluster assignments against true grape cultivars
bottle_cluster <- som_clusters[wine_som$unit.classif]
confusion_matrix <- table(True_Cultivar = vintages, SOM_Cluster = bottle_cluster)
print(confusion_matrix)
```

6 Homework Phase 5: Supervised SOMs (`xyf`) & Quality Metrics (`aweSOM`)

When class labels or multiple data blocks are available during training, `kohonen` provides two multi-layer extensions:

- `xyf()` **(Supervised X-Y Fused SOM):** Jointly maps predictor matrix **X** and target matrix **Y** (encoded via `classvec2classmat()`), allowing neurons to organize around a weighted blend of feature similarity and class membership.
- `supersom()` **(Multi-Layer SOM):** Accepts a named `list()` of matrices (e.g., chemical spectroscopy, soil mineral composition, and sensory scores) with custom layer weights.

Furthermore, the `aweSOM` package provides rigorous quantitative benchmarks via `aweSOMquality()`:

1. **Quantization Error (`err.quant`):** Mean Euclidean distance from each observation to its BMU codebook vector (lower is better).
2. **Topographic Error (`err.topo`):** Fraction of observations whose first-best matching neuron and second-best matching neuron are *not* adjacent on the grid. Values near 0 confirm an untwisted, topology-preserving map.
3. **Explained Variance Ratio (`err.varratio`):** Percentage of total dataset variance captured by the codebook vectors (higher is better).

```r
set.seed(42)

# 1. Train a Supervised SOM (xyf) fusing chemistry (70%) and cultivar class (30%)
wine_xyf <- xyf(
  X = wines_scaled,
  Y = classvec2classmat(vintages),
```

```
  grid = somgrid(5, 5, "hexagonal"),
  rlen = 500,
  user.weights = c(0.7, 0.3)
)

# Predict the cultivar of the last 10 bottles using only their chemical features
preds <- predict(wine_xyf, newdata = list(X = wines_scaled[168:177, ]))
data.frame(
  Actual = vintages[168:177],
  Predicted = preds$predictions[[2]]
)

# 2. Evaluate quantitative map quality metrics via aweSOM
qual_metrics <- aweSOMquality(som = wine_som, traindat = wines_scaled)
print(qual_metrics[c("err.quant", "err.varratio", "err.topo")])

# 3. Generate an interactive circular radar map in RStudio Viewer
aweSOMplot(
  som = wine_som,
  type = "Circular",
  data = wines,
  variables = c("alcohol", "flavanoids", "proline", "acidity", "col.int"),
  superclass = som_clusters
)
```

7 Autonomous Work

Building on the techniques covered in the guided walkthrough, you will now analyze a real-world dataset sourced from the UC Irvine Machine Learning Repository. For this autonomous investigation, filter the repository using the following criteria:

- Data Type = Multivariate
- Task = Clustering or Classification (containing at least $N \geq 150$ observations)
- Feature Type = Real / Numeric (at least $p \geq 6$ continuous attributes)

Select **one** dataset meeting these criteria (excluding the Italian Wine dataset used above) and complete the following four analytical tasks:

1. **The "Unscaled Bully" vs. Standardized SOM Experiment:** Train two 5×5 or 6×6 hexagonal SOMs using `kohonen::som`—one on the raw, unscaled numeric matrix and one on the standardized (`scale()`) matrix. Compare their `type = "codes"` plots and U-Matrices to demonstrate how high-variance features distort unscaled maps.
2. **Topological Architecture Benchmark:** Using your standardized dataset, train and compare four grid configurations across `topo = c("hexagonal", "rectangular")` and `toroidal = c(FALSE, TRUE)`. Report a summary table comparing Quantization Error (`err.quant`), Topographic Error (`err.topo`), and Explained Variance (`err.varratio`) computed via `aweSOM::aweSOMquality()`.

3. **Component Planes & Two-Stage Hierarchical Clustering:** For your best-performing hexagonal SOM, generate at least four informative single-variable heatmaps (`type = "property"`), apply Ward's hierarchical clustering (`ward.D2`) to the codebook vectors, and plot the segmented map with `add.cluster.boundaries()`. If your dataset includes an external class label, compute and discuss the confusion matrix comparing unsupervised SOM clusters against the ground-truth labels.

4. **Supervised Fusion (`xyf`) or Mystery Sample Projection:** Either train a supervised `xyf()` network using an outcome variable and evaluate its test accuracy via `predict()`, or construct two synthetic "mystery profiles" (e.g., extreme high/low feature combinations) and project them onto your trained SOM using `map()` to identify which cluster tribe claims them.

8 Deliverables

Just have fun!

Happy Hacking!